

WSUDAS-AI

Final Report

WSUDAS



Mentor:

Parteek Kumar

WSUDAS-KS

Kieran McHugh, Son Nguyen

CptS 423 Software Design Project II

Spring 2026

TABLE OF CONTENTS

I. Introduction	4
II. Team Members	5
III. Project Requirements Specification	6
III.1. Project Stakeholders	6
III.2. Use Cases	6
III.3. Functional Requirements	8
III.4. Non-Functional Requirements	9
III.5. Standards	9
IV. Software Design	9
IV.1. Architecture Design	10
IV.1.1. Overview	10
IV.1.2. Subsystem Decomposition	11
IV.2. Data Design	14
IV.3. User Interface Design	15
IV.4. Constraints	17
V. Test Case Specifications and Results	17
V.1. Testing Overview	17
V.1.1. Unit Testing	18
V.1.2. Integration Testing	18
V.1.3. System Testing	19
V.1.4. Functional testing:	19
V.1.5. Performance testing:	19
V.1.6. Standards and Constraints Verification / Testing:	20
V.1.7. User Acceptance Testing:	20
V.2. Environment Requirements	20
VI. Projects and Tools used	20
VI.1. Languages	20
VI.2. Frameworks and Libraries	20
VII. Description of Final Prototype	21
VIII. Social Responsibility and Broader Impacts	22
IX. Product Delivery Status	23
X. Conclusions and Future Work	23
X.1. Limitations and Recommendations	23
X.2. Future Work	23
XI. Acknowledgements	24
XII. Glossary	25
XIII. References	25
XIII. Appendix A - Team Information	26
XIV. Appendix B - Example Testing Strategy Reporting	26
XV. Appendix C - Project Management	27

I. Introduction

Agricultural institutions and farmers are responsible for producing the food we eat every day, and the agricultural industry is a major component of the U.S. economy, accounting for over a trillion dollars of the United States' G.D.P. [1]. Because it is such a large and valuable industry, development and utilization of more efficient practices can save farmers (and in turn consumers) hundreds of millions of dollars.

Washington State University Decision Aid System [2] is a service designed to help farmers and other agricultural institutions predict various factors that affect their crops, ranging from weather, what insects are present, what the populations of those insects are, and when pesticides should be applied to protect crops. Large quantities of data are collected from a multitude of sensors and processed by DAS. Users can then choose nodes and use the data collected at those locations to better inform their decisions and practices.

While DAS has proven very effective in aiding farmers, it was originally tailor-made specifically for informing farmers growing potatoes, but its scope has been expanded much further beyond that over time. The data being collected now applies to a multitude of other crops and is utilized by many growers across the region.

We were put in touch with David Crowder of WSUDAS and worked with the organization for both the Fall 2025 and Spring 2026 semesters. Both semesters had separate goals: The first was to integrate a third party trap system (Scoutlabs traps) into their existing contour system. The second semester's focus was providing access to the NYUS2.2 freezing tolerance model for grape growers.

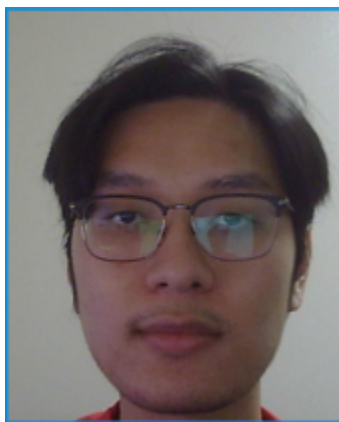
II. Team Members

II.1. Kieran McHugh



Kieran McHugh is a senior majoring in computer science at Washington State University. His interests include machine learning, artificial intelligence, and data science. Kieran has worked with Python, R, C/C++, and HTML, and has worked with the Amazon Web Services platform. For this project he primarily focused on writing the data transfer for Scoutlabs traps as well as converting the feature extraction for the NYUS2.2 model.

II.2. Son Nguyen



Son Nguyen is also a Senior at Washington State University, majoring in computer science. His coursework shows a slight interest in data science, machine learning related topics, and a slight interest in cyber security. Son has some experience in Java, Python, C#, and SQL queries. For this project, he works mostly on similar topics to Kieran.

III. Project Requirements Specification

III.1. Project Stakeholders

The stakeholders for this project are the faculty of WSUDAS as well as their user base. WSUDAS wants to be able to integrate third party Scoutlabs traps into their contour system and their users want to be able to see more accurate, realistic, and up-to-date information on WSUDAS' site so that they can make more informed decisions about when and whether they should apply pesticides to their crops. With the addition of the NYUS2.2 grapevine freezing tolerance model in January 2026, grape growers (primarily in the British Columbia region) are also another stakeholder in the project as well.

III.2. Use Cases

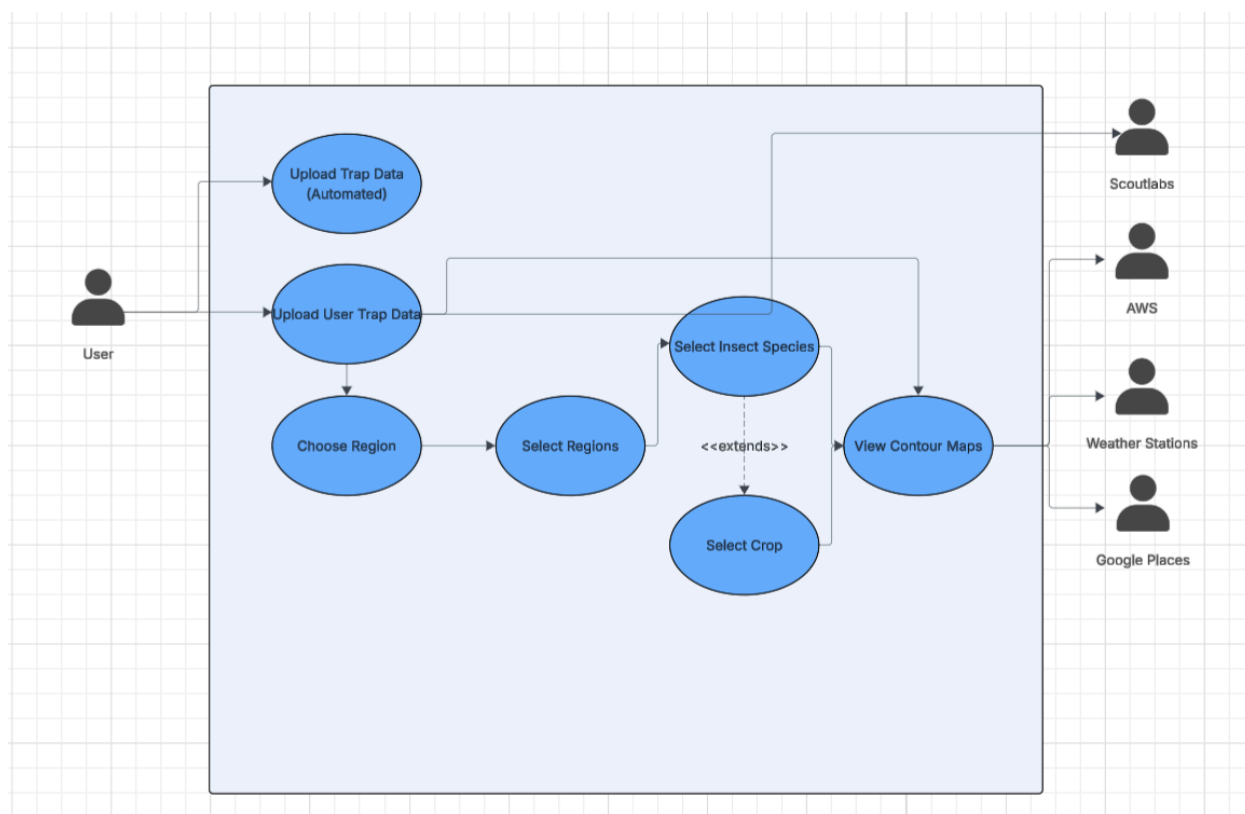


Figure 1: Use Case Diagram

Figure 1: Describes how a typical user would interact with the WSUDAS website

- UC-1: Trap data is uploaded to WSUDAS
 - Actors: Customer/User with Scoutlabs Trap
 - Precondition: Trap is connected to WSUDAS' database
 - Postcondition: Trap data will automatically appear in WSUDAS' database
 - Basic Path:
 - User sets up their Scoutlabs trap and connects it to WSUDAS

- When the trap is ready to upload data, the data is automatically pushed to WSUDAS
 - Data is cleaned and entered into the model/contour system
 - Alternative Path:
 - (Also the current system) User uploads .csv file manually to WSUDAS' database
 - Data is automatically cleaned.
 - Related Requirements:
 - Connection to WSUDAS
 - Scoutlabs API integration
- UC-2: Scoutlabs Data Included in Contours
 - Actors: User/Developer
 - Precondition: Scoutlabs traps have uploaded their data to WSUDAS (either manually or automatically)
 - Postcondition: Insect data from the Scoutlabs traps is used when generating contours
 - Basic Path:
 - User logs into the DAS website
 - Scoutlabs trap data has been sent to DAS (either by them or other users)
 - User navigates to the contour maps and selects an insect
 - Insect data and contours displayed to the user include the data uploaded by the Scoutlabs traps
 - Related Requirements
 - The user (if not a developer) has subscribed to WSUDAS
 - Scoutlabs traps have uploaded insect data
- UC-3: NYUS2.2 Model Executes Using Weather Data
 - Actors: User/Developer
 - Precondition: Scoutlabs traps have uploaded their data to WSUDAS (either manually or automatically)
 - Postcondition:
 - NYUS2.2 model output is generated and stored in the Model Service database
 - Model results are available for contour generation and frontend display
 - Basic Path:
 - Data is collected by weather station and stored in Weather Service
 - Data is processed through feature extraction and fed into the NYUS model
 - The system executes the NYUS2.2 model using the prepared input data
 - Model output is generated and stored in the service-models database
 - Related Requirements
 - Weather Service integration
 - Model Service database integration
 - NYUS2.2 model execution environment
 - Scheduled transfer job system
- UC-4: UserS Views the NYUS 2.2 Model Results
 - Actors: User/Developer
 - Precondition:
 - NYUS 2.2 model is successfully executed

- User has access to DAS website
- Postcondition:
 - User can view contour maps and model predictions generated by the NYUS2.2 model
- Basic Path:
 - User logs into the DAS website
 - NYUS 2.2 output has been sent to DAS
 - User navigates to the contour maps and selects grapes and NYUS 2.2 model
 - The result is displayed
- Related Requirements
 - Weather Service integration
 - Model Service database integration
 - NYUS2.2 model execution environment
 - DAS frontend visualization system

III.3. Functional Requirements

ID	Requirement Name	Description	Source	Priority Level
FR-01	Automated Data Ingestion	The system must be able to collect data automatically from the smart traps with minimal human intervention	The End User	0
FR-02	Smart Traps Integration	Going from FR-01, the system must be integrated with smart traps APIs, namely Scoutlabs, to automatically collect and digest data for future usage. It should be able to throw errors and notify the developers if something goes wrong	WSUDAS Team	1
FR-03	Contour Map Generation	The System must be able to generate contour maps based on the data provided by the traps. If trap data is missing, it must still be able to generate maps based on available data	WSUDAS Team	0
FR-04	Unified Spatial-Temporal Model	The system must unify both the time and space models into one model to provide more precise predictions	WSUDAS Team	2

FR-05	Insect Abundance on Map	The system must render accurate insect abundance on the existing map	End User	1
FR-06	NYUS2.2 Model Output	Users should be able to see NYUS2.2 model output in some form	End User	2

III.4. Non-Functional Requirements

Non-Functional Requirement-01: The data collection automation part must be simple enough for normal users.

Non-Functional Requirement-02: The models should be functional on more than 1 treefruits.

Non-Functional Requirement-03: The system should be scaled to work on at least 3 different locations without interfering with one another.

Non-Functional Requirement-04: The suggestion provided by the system should be at least 1 week earlier for better planning.

Non-Functional Requirement-05: The system should be able to update information daily for better accuracy.

Non-Functional Requirement-06: The NYUS2.2 model's python implementation of the feature extraction step should perform almost identically to the original R implementation.

III.5. Standards

Our project adheres primarily to IEEE 1016 standards. Because we are building off of an existing system, many design changes have already been made for us. As such, we must work around the existing architecture and are somewhat more limited in how we can approach problems. For this reason it is important for us to document the methodology and rationale of our approach in addition to the implementation itself, as what might seem inefficient at face value may be required to work properly within the system. Additionally, IEEE 1016 standards promote looking at the project through multiple perspectives, and as our user base is primarily those in the agricultural industry, we need to ensure that what we produce will be both useful and usable in their daily lives.

IV. Software Design

IV.1. Architecture Design

WSUDAS' purpose and goal is to allow users to view agricultural information within a user-defined area. The user selects stations which are then used as points on a map. The user's station selections are then cross-referenced with the crop and insect data within the WSUDAS database and then both are used to generate the contours the user will see. These contours are drawn over the map and allow the user to see what the status of crops and pests are in their specified regions of the map. This is accomplished by using a microservices architecture to link multiple systems and services together, including weather stations, Amazon

Web Services, and individual farmers. The NYUS2.2 implementation does not build off of any existing architecture within WSUDAS and requires some additional setup to get running on WSUDAS' existing architecture. For us this meant setting up a new docker container to run the LT50 model and refactoring some of the model's existing components to integrate smoothly with the system's components.

IV.1.1. Overview

For the structure of our project we are somewhat limited by the constraints of the existing system. Making even seemingly small changes to the overall design schema would require refactoring enormous portions of the existing project and is therefore not desired by the client. As such, any design decisions we make must also adhere to the current structure of WSUDAS' services and cannot conflict with or significantly change them. The only exception to this is the addition of the NYUS2.2 model (as mentioned directly above in the Architecture Design section).

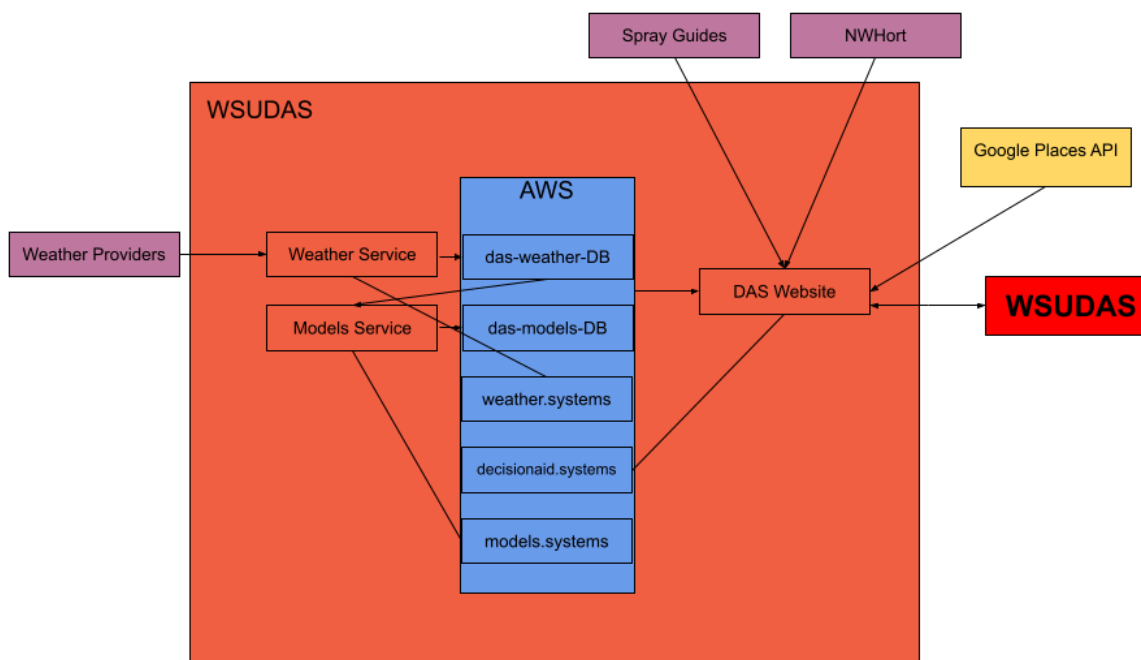


Figure 2: Block Diagram of DAS System

In the above model, **Weather Providers** give the basis for data used in WSUDAS' system. This data is then fed into the **Weather Service** component which then transfers the data to the **Amazon Web Services databases** where this data is stored. The **Models Service** then utilizes this data to generate the models the user sees. These models are then transferred to another database (**Models DB**) and are then propagated to the **DAS Website**. Utilizing some additional third party software, these models can provide users with information on when to make sprays as well as show a map of the region (using **Google Places/Earth** [9]), and this map and models then show the generated contours to the user.

IV.1.2. Subsystem Decomposition

Since this is already an existing system, it can be broken down into three main subsystems: the weather services, the model services, and the website. While each is a separate system, they all rely on one another and utilize similar services when deployed.

III.2.1 Weather Service

- a) Description: Ingests weather observation from appointed sources, validate and pre-processing the data
- b) Concept and Algorithm Generated:
 - i) ELT and possible normalization: data provided would be cleaned, transformed, and normalized before getting saved into the database
 - ii) Schema design: the schema design and mapping of data collected
- c) Interface description:
 - i) Service required: Data Storage (AWS), Weather station for data collection, Database (MySQL)
 - ii) Service provided:
 - 1) Cleaned Data to Model Service for further analytics. The input would be raw data from stations and traps. The output is processed data.
 - 2) Data to Web Service for displaying basic information. The input is cleaned data from the first step, the output is data successfully loaded into database for future usage

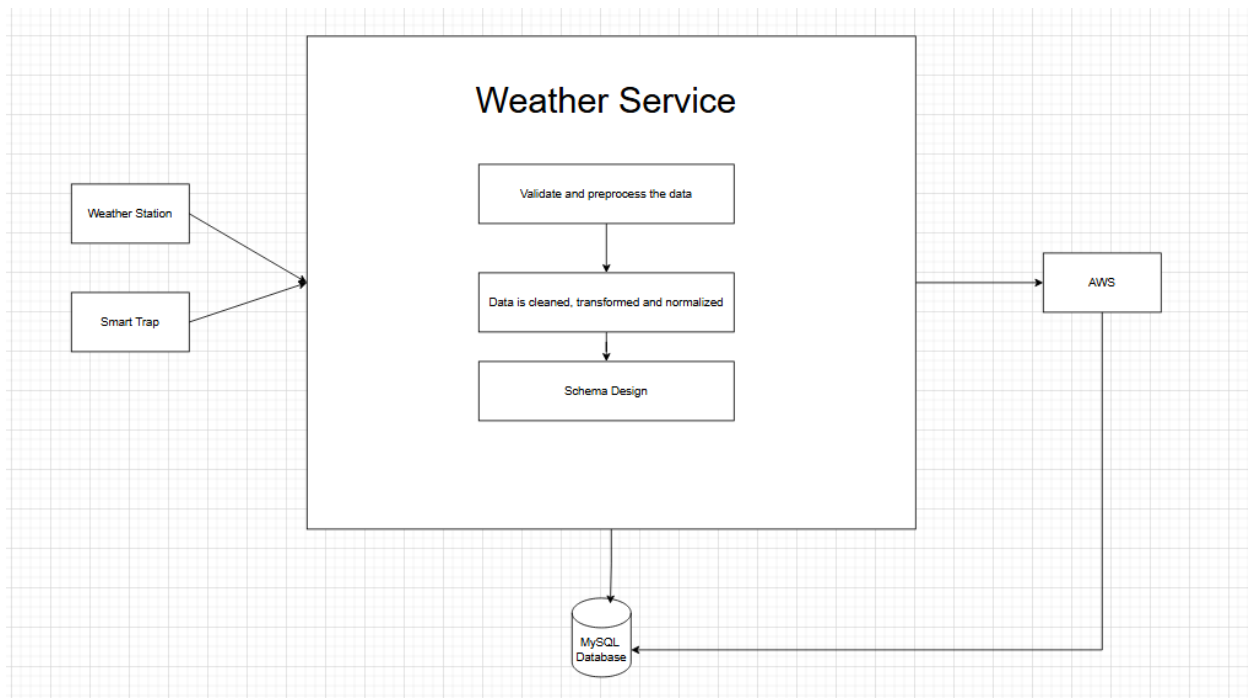


Figure 3: Weather Service Visualization

III.2.2 Model Service

- a) Description: Run models based on the data provided by the Weather Service, produces contour features, writes outputs to store in the database and publish it to the website
- b) Concept and Algorithm Generated:
 - i) Model design: Design decision models for spraying, insects development, and other related factors
 - ii) Spatial interpolation: configure contours for regions with preset tailored to them
- c) Interface description
 - i) Service required: Data Storage (AWS), Database(MySQL), Weather Service for data, Google Place
 - ii) Service provided:
 - 1) Contour maps in GeoJSON to Website for showing the location applied to the suggestion. The input is cleaned data from the database, the output is GeoJSON file helping generating the contour maps
 - 2) Suggestion to Website for giving recommendation for users to follow. The input is the cleaned data and the GeoJSON file, the output is suggestion based on the cleaned data from the past til present and the location

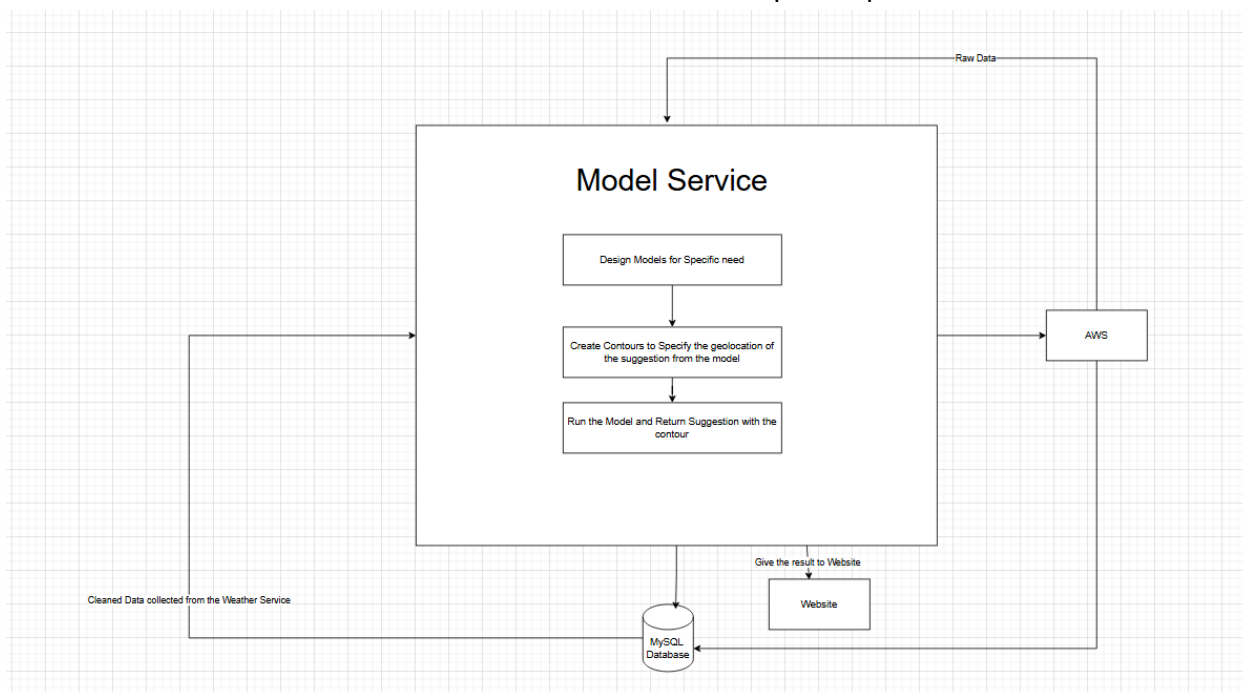


Figure 4: Model Service Visualization

III.2.3 Website

- a) Description: User-facing website that visualize the models output as contour maps, show station information, and provide suggestion
- b) Concepts and Algorithms Generated:
 - i) Map UX: Provide a map based on Google Places/Maps, layer toggle, time sliders
 - ii) Accessibility: color scales for contours, basic data visualization
- c) Interface description:
 - i) Service required: Weather Service for station basic data, Model Service for contours and suggestion, Authentication for login, Google Places/Maps for map and geocoding
 - ii) Service provided:
 - 1) Application to end user. The input would be selecting the treefruit, type of suggestion and location. The output is visualized data and suggestions.

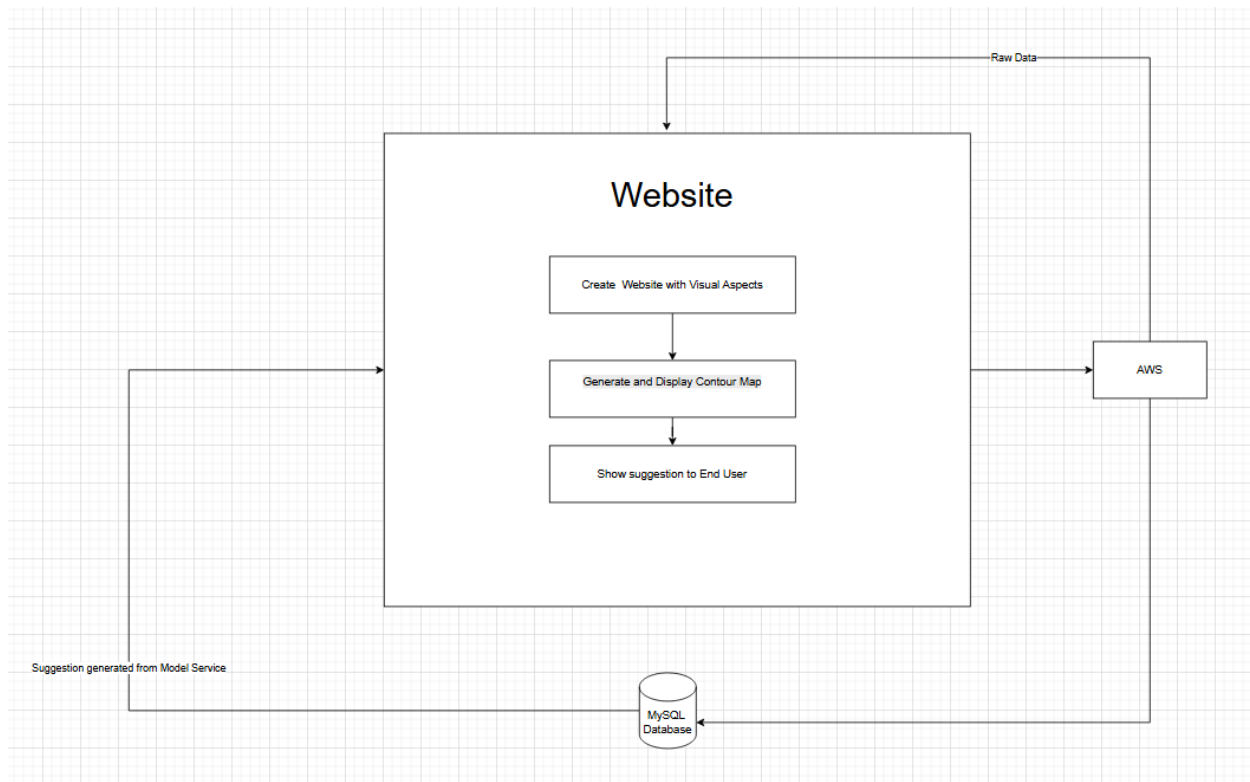


Figure 5: Website Service Visualization

IV.2. Data Design

The project has numerous data structures and uses a large and varied database to fulfill its functionality. The database contains entries for users, stations, and regions, which are stored when the user enters said information into the WSUDAS website. Users can customize and update these values to view data from their preferred stations. Insect, larva, and egg data is then used to determine when each pest will reach certain stages in their development, helping those in the agricultural sector decide whether and when they should take action to mitigate the potential damage those pests could cause. Users who opt to can also upload their data to aid the accuracy of the models, and all insect data is dated to allow users to look at the past to help them make better decisions in the present.

On the developer-facing side, the contours generated by the user's input are stored in the database with a unique ID and are associated with the stations used to generate them. These contours are stored in a PHP class structure with the associated stations being member values, and various functions are used in the process of retrieving and displaying the data to the user. However, the vast majority of the data associated with the project is simply stored in a MySQL database and is retrieved and displayed to the user when prompted. Data used by the NYUS2.2 model can be taken from WSUDAS' existing database since there is already more than enough data for grape temperatures, so no additional efforts are needed to integrate the model in this respect besides sampling the grape records from the database.

As for the storage of the data itself, numerous services are involved in gathering, storing, and retrieving the data. This includes AWS (specifically AWS S3 buckets [6]), Google Places, and independent weather providers. Weather providers feed their data to DAS, which stores it using AWS services, and location data can be provided using Google Places. The following diagram depicts the system's data storage:

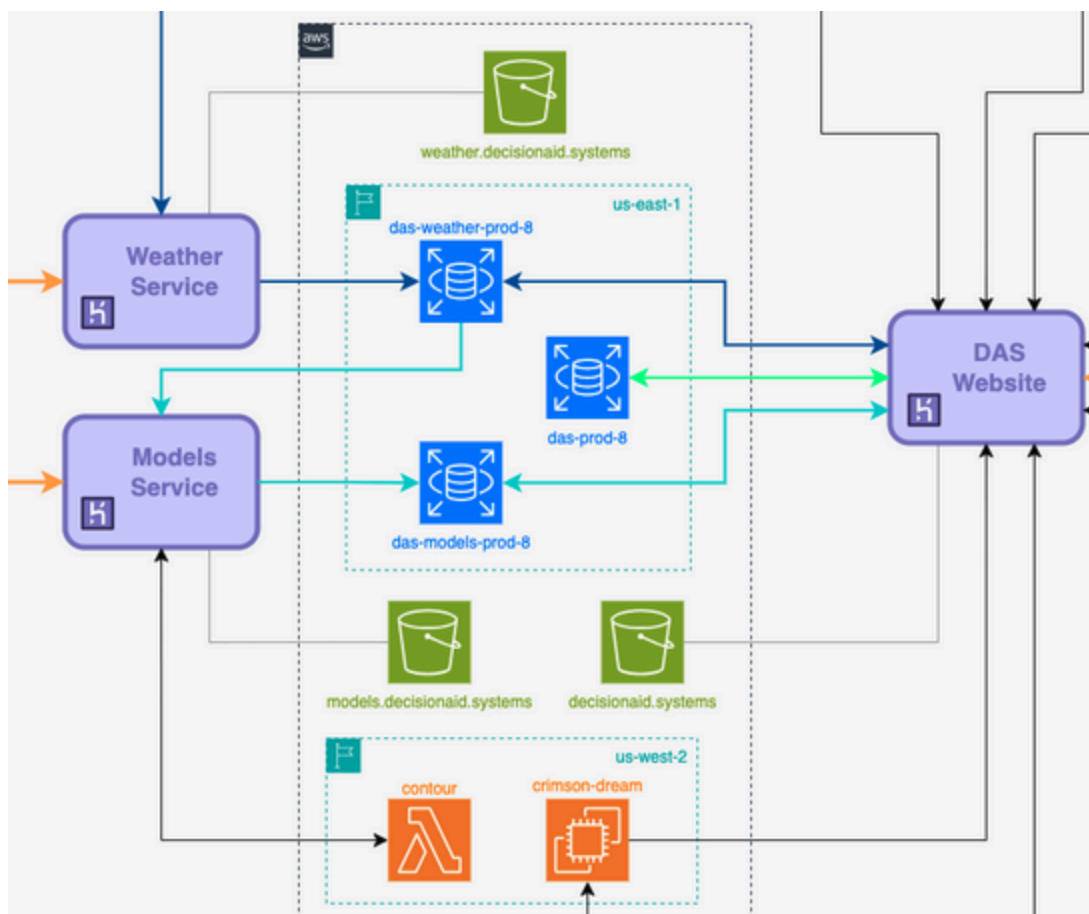


Figure 6: Data Design of WSUDAS

When generating contours, some data manipulation is involved. Each station acts as a point and has data for various insects associated with it. When generating a contour, the data for each station is gathered and then essentially “spread out” over an area depending on the density of each insect population at each station (The resulting contour often looks similar to Perlin noise generation with added variation to smooth out the visual disparities). The contour maps communicate the target region to the user through the shaded areas and then the densities of each insect’s population depending on the difference and intensity of the contour’s colors.

IV.3. User Interface Design

The user interface is viewed through WSUDAS’ site and gives the user the choice of what region they would like to see data from, what specific data they would like to view, and then displays a contour map given this input. The contours the user sees will only use the data from the stations they have specified. Users then choose what crops they would like to view (tree fruits, potatoes, etc) or which pests (each species’ larva, eggs, and adults). Since data is stored with dates in WSUDAS’ system, users have the ability to look at historical data using a date slider which can be especially useful when planning for the next season.

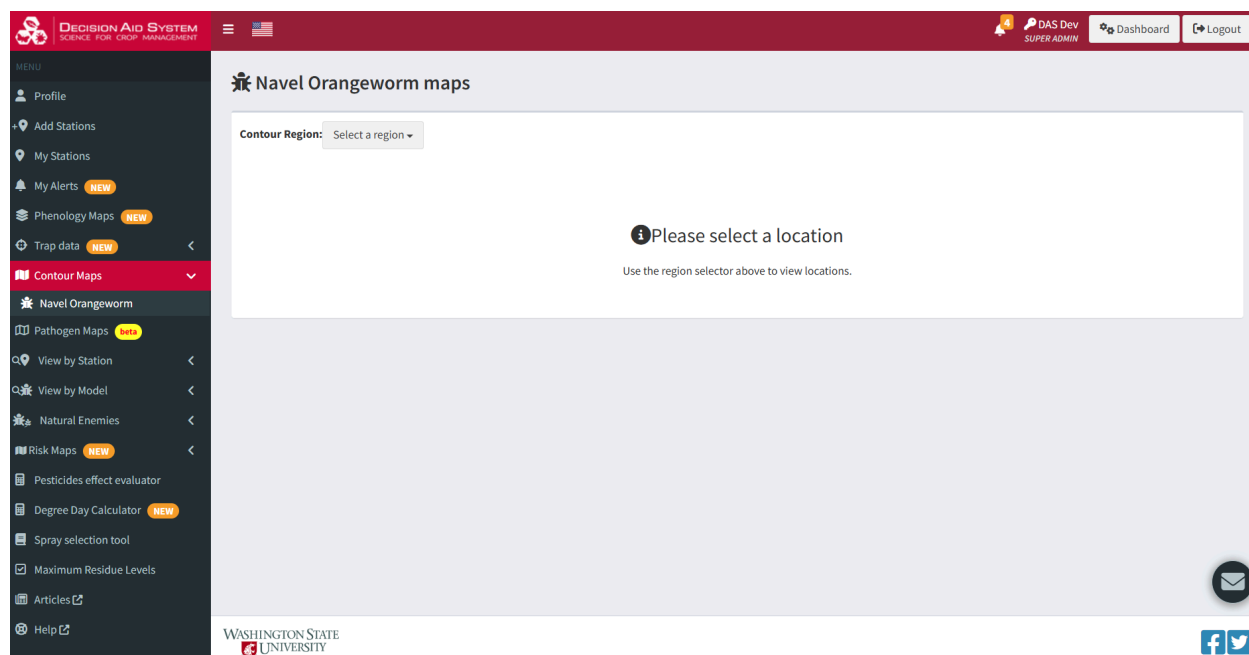


Figure 7: User-facing Region Selection for Navel Orangeworm (User Perspective)

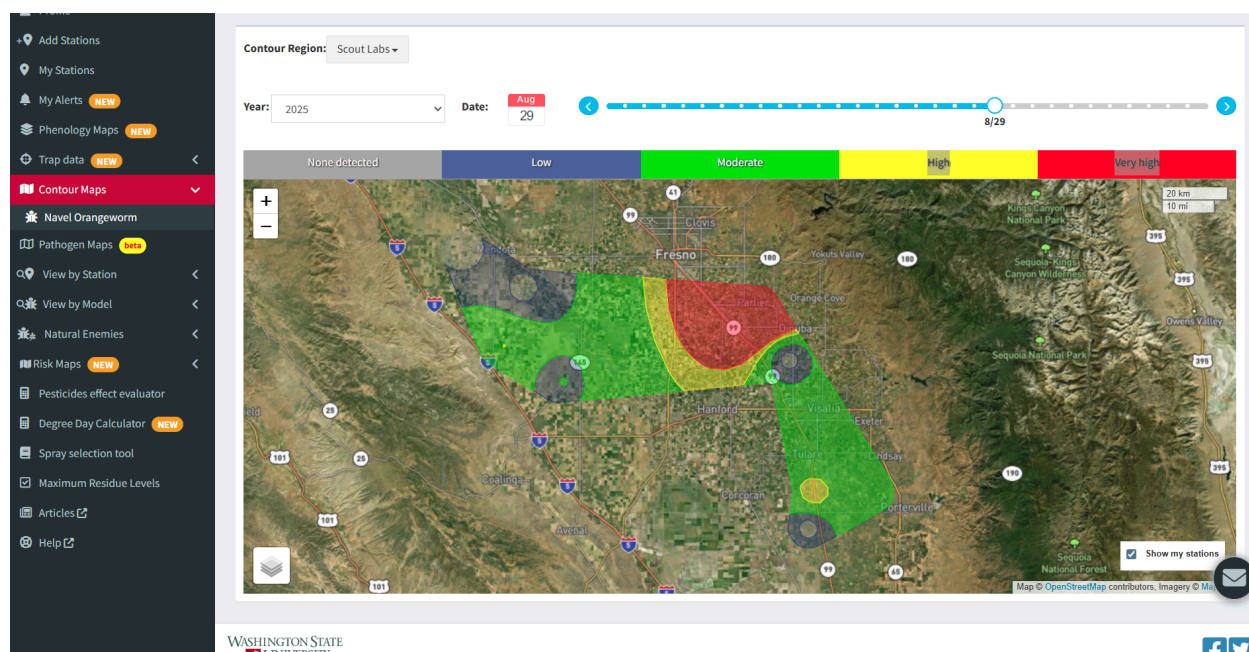


Figure 8: Sample Generated Contour Region (User Perspective)

Since our project is web-based, the only installation required on the user-end is the station/traps setup if required. A typical user flow would be logging into the website to gain access to the website's full functionality (this functionality is already complete and is thus not within the scope of our project), choosing their region, choosing the type of treefruit and pests for viewing, and uploading their data in .csv format. The rest is taken care of in the backend. Then the contour

region is generated and displayed to the user, allowing them to make decisions based on what the predicted insect populations are nearby. Once our data transfer component is complete, our goal is to make it so that users who have opted in to WSUDAS' system with their Scoutlabs traps will have their data automatically entered into the system and will thus be able to view their data without the extra step of uploading it.

The NYUS2.2 integration will not have a dedicated user interface component since there is simply not enough time in one semester for the two of us to fully integrate this component in WSUDAS' system. However, we will provide the developers the ability to use the model in its entirety and integrate it within the existing system, allowing them to build out the user-facing side of it should they wish to.

IV.4. Constraints

There are numerous constraints to take into consideration when working with the DAS system. As it is an existing live-service system, we do not have too much freedom to make changes to the overall structure since that would result in unnecessary downtime and relearning of the codebase which is something the client wishes to avoid. Furthermore, the Scoutlabs traps we are working with output data in a standardized format which is helpful, however, we are unable to change what data we are receiving exactly. Therefore we must work around these conditions when using the Scoutlabs data. When testing we need to keep these constraints in mind as there may be barriers we have to work around rather than through (like a disambiguation error we ran into when using the Scoutlabs trap data).

During the exploration phase of the NYUS2.2 model there were some additional constraints to take into consideration. For instance, we first needed to evaluate whether the model itself would be a reliable and worthwhile addition to WSUDAS, which involved passing sample data through the feature extraction process and model and then verifying that the data was both consistent with the provided data values and provided useful information WSUDAS could display to users. Furthermore, we also needed to consider how the model would integrate with the website (since the model itself uses Python and R code while WSUDAS' existing infrastructure is almost entirely PHP). An API endpoint to the autogluon model is the most feasible solution at this point in time.

V. Test Case Specifications and Results

Our planned testing strategy will primarily incorporate a CI pipeline, as continuously deploying code without verification of its proper functionality is not acceptable for our client (they are running a live service and thus cannot afford to spend the time needed to roll back their site due to faulty code). With this method our code will be automatically tested when pushed while still giving us the flexibility to postpone certain elements that are not yet working without requiring a full reboot of the system.

V.1. Testing Overview

The majority of our work over the course of this project has consisted of transferring data between existing locations within DAS' database, the majority of our testing involves ensuring that the correct values are moved to the right locations in the correct format. This necessitates having an understanding of the relationship between tables in the databases, the relationship between databases of different services, and making sure that each step of the transfer is grabbing the correct information before committing any changes to the database. As such, we have been taking steps to verify that each step in this process produces the expected results.

For the NYUS2.2, the aim is for verifying the accuracy of the new model, following the same flow as the data transfer in PHP. Data is pulled from the Weather Service and then fed into Model Service and displayed on DAS website. The only difference is the verified part now changed from the validity of the data transferred into Model Service to comparing the result of the new model to some existing one.

V.1.1. Unit Testing

As per the norms of unit testing, we will pass any standalone functions through a unit test to ensure that they are performing as we expect in an environment where they will not have an impact on the database should something unexpected occur. Since most of our methods currently involve grabbing data from one table and transferring it to another, there is not much to test during this process besides ensuring that the correct data appears where we expect it to. One way we could do unit testing is by doing equivalence class testing, where we'd try to test sample datasets falling into some notable category like all duplicates, non duplicates and so on.

Unit testing for the NYUS2.2 model integration focuses on validating individual components involved in preparing inputs and handling outputs. This includes testing functions responsible for extracting weather data, formatting inputs into the structure required by the model, invoking the model execution process, and storing the model results in the database. We will also test edge cases such as missing data, invalid values, and boundary conditions to ensure that the model integration handles these situations gracefully without causing system failures or incorrect outputs.

For the R to Python feature extraction conversion we tested our data by running our converted script against the sample data provided and checked for any discrepancies between the two. To confirm this we wrote a short python script to compare each corresponding value in both the original feature extraction and our new implementation, highlighting any rows where there were significant differences. This made us aware of an issue calculating GDH values in our first implementation of the feature extraction. After addressing this we found that our version was functionally identical, being within 1/100th of a degree when run through the NYUS model (the slight difference being due to how Python handles rounding decimals versus R. This difference is negligible in the project's context as a grower cannot reasonably account for it). In this way we were able to guarantee that this step of our project was working correctly.

V.1.2. Integration Testing

Integration testing has been an important aspect of our project thus far, as there are numerous individual, compartmentalized pieces in DAS' system. The database is split across the docker containers, each containing dozens of tables used to store the data the system depends on. As such, it is important to guarantee that these separate pieces interact correctly with each other.

Integration testing for the new model ensures that all components in the data pipeline work together correctly, including the Weather Service, Model Service, database containers, and frontend visualization components. These tests verify that weather data flows correctly into the model, that the model executes successfully within the containerized environment, and that its outputs are correctly stored and retrieved. We also confirm that model results are properly associated with the correct locations, dates, and trap data, ensuring consistency across all services involved in the DAS system.

V.1.3. System Testing

For what we have produced so far, black box testing would be directly comparing data processed with expected data. The tables in service-weather would be populated with sample data and the data inserted into service-models will be put into a test-case to compare with expected data. The whole DAS system is an extensive project with many parts we don't have to work on, so a full black box testing is deemed unnecessary. During the black box test, the first action is syncing the trap data into location, then, the location would be used as a reference point for syncing the records into the trap counts, the result would be two fully populated table locations and trap counts. We can also validate this on the front end using automation libraries like Selenium(8) to test out a basic system testing mimicking an end-user's flow. Going from updating the data on the front-end, double checking the data validity, and seeing if the contours are generated as expected.

System testing now includes validating the full end-to-end workflow involving the NYUS2.2 model. This begins with weather data being ingested into the system, followed by model execution, storage of model results, and presentation of those results on the DAS website. We verify that model outputs appear correctly on contour maps and other visualizations and that they accurately reflect the input data. Automated testing tools such as Selenium may also be used to simulate user interaction with the frontend to confirm that model-generated data is properly displayed and accessible.

V.1.4. Functional testing:

Functional testing would mean testing the functionality of the program. For our case, it would mean testing the first data transfer, the referencing for the second data transfer, and the second data transfer. At each step we verify that the data is of the correct type and value during the transfer.

Functional testing will ensure that the NYUS2.2 model executes correctly when triggered and produces valid outputs based on the input weather and trap data. This includes verifying that model execution functions are called at the appropriate time, that input data is correctly passed into the model, and that outputs are stored in the correct database tables. We also verify that the frontend correctly retrieves and displays these outputs, ensuring that users can access accurate model predictions through the DAS website.

V.1.5. Performance testing:

We have already begun design implementations that keep stress-testing in mind, which includes using chunked inserts to the database so that the service does not run out of memory. We also check to see that our inserts run in a reasonable amount of time. The stress test is also minimized by the impact of chunking, so having one is not needed.

V.1.6. Standards and Constraints Verification / Testing:

Standards and Constraints Verification focuses on ensuring the data transfer is acceptable under DAS's technical standard. We validate database constraints like Primary/Foreign keys, Not Null, uniqueness and so on. Domain rules like no negative trap counts, valid date ranges, proper date increment is also used. Things like execution times and chunk size are also kept in check.

V.1.7. User Acceptance Testing:

Since our work is mostly done on the back end of the project, UAT cases for this project would be having users go into the website and try out the features, which is not limited to what we work on. The users would go to the website, try to create the location and insert the trap data manually, and let the system do the work. The result is a contour map with the data added from Scoutlabs traps.

V.2. Environment Requirements

The testing environment requires access to WSUDAS' system through local docker containers that can run the system separately from the main site. Additionally, we use php's unit testing, of which the libraries are built-in. NPM also needs to be installed and run through the web container to properly view the frontend of the site for any testing that must occur there. Using the NYUS2.2 model requires access to WSUDAS'

VI. Projects and Tools used

VI.1. Languages

PHP	BASH Shell	Python	R
-----	------------	--------	---

VI.2. Frameworks and Libraries

Tools/Framework/Library	Usage in Project
AWS S3	Data storage for weather and model data
Laravel (PHP)	Backend data transfer and services, also used as a MySQL query replacement with the built-in queries builder
Docker	Containerized deployment
AutoGluon	Machine learning model (NYUS2.2) implementation

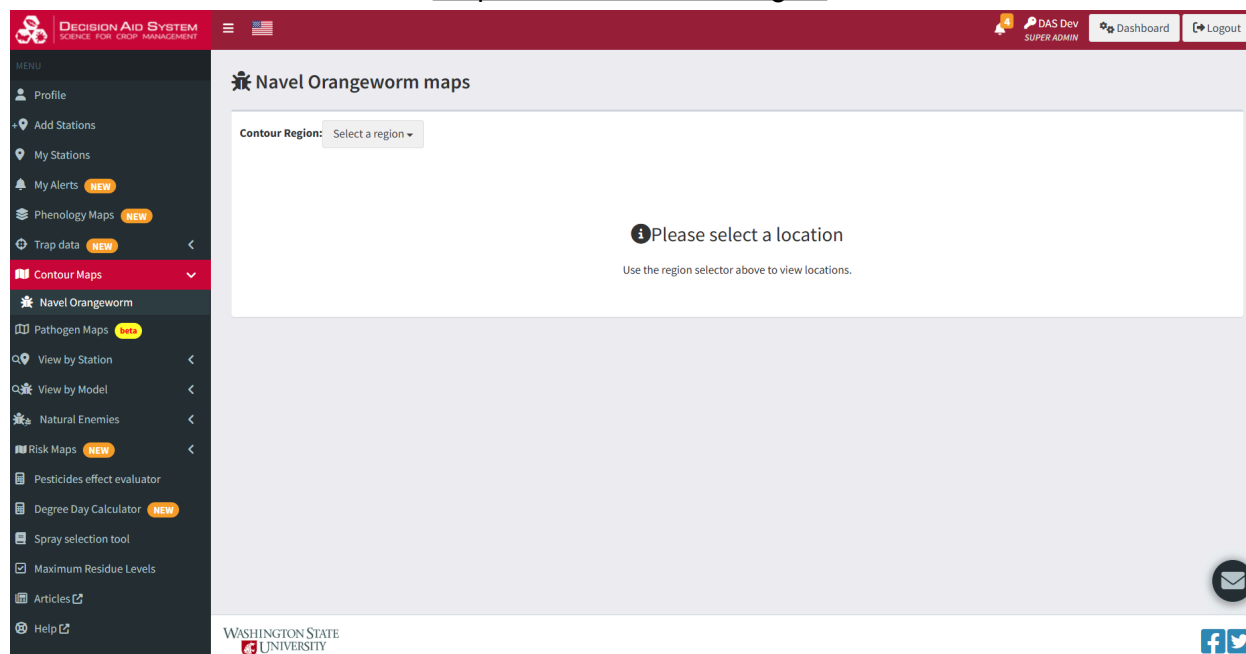
Python (Panda, Numpy)	Model execution and Data Transfer
Google Maps API	Visualization of contour maps
Python (rpy2), R(ggpubr, weathermetrics, dormancyR, chron, chillR)	Data extraction in R and conversion to Python
Python (FastAPI, Uvicorn)	API implementation of the model as a service

VII. Description of Final Prototype

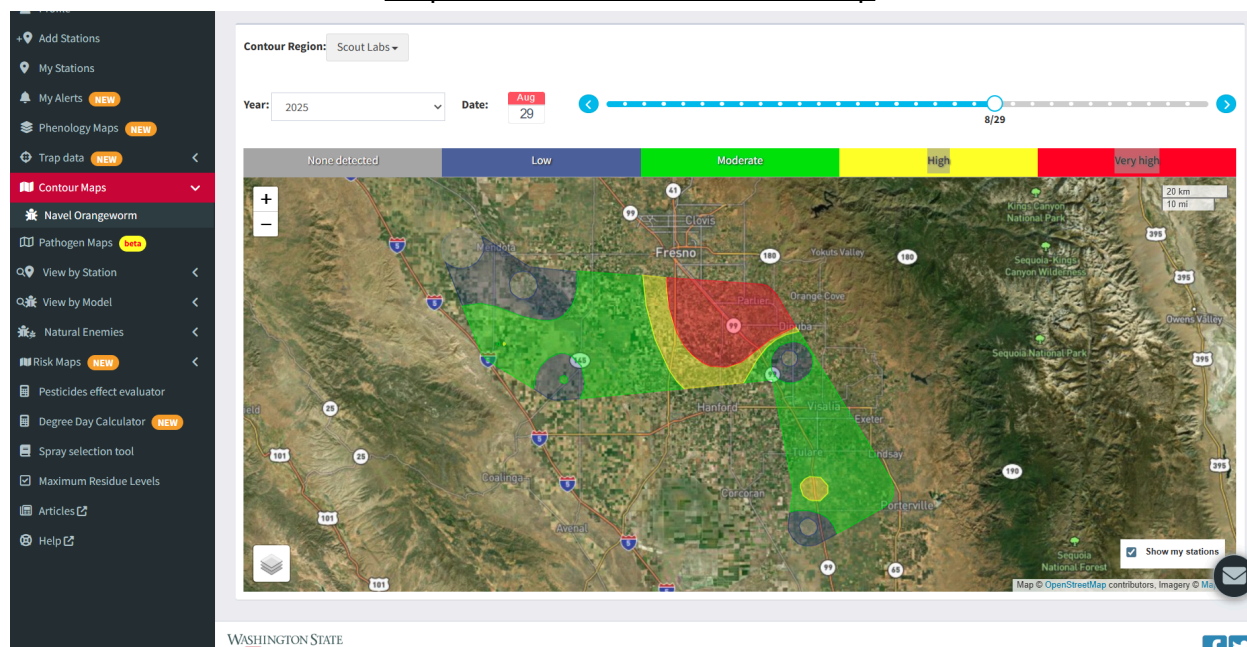
Because our project consists of two separate major milestones (Scoutlabs traps integration and our implementation of the NYUS2.2. model) there are two separate final prototypes to consider. From a user-facing perspective, the work completed with the Scoutlabs traps was done specifically so that the user does *not* have to go through additional steps to use the system and instead removes the need for them to manually upload their trap data since that is now being done automatically. As such, the normal user flow for this process remains largely unchanged besides the removal of manual uploading. As for the NYUS2.2 model, our goal is to make it usable for the client on their end and ready to deploy to users, though we will not be involved in the deployment of it ourselves. It will be up to WSUDAS to decide when and how to proceed with our work from there.

VII.1. Scoutlabs User Flow

Step 1: Select Desired Region



Step 2: View Generated Contour Map



Once the desired region and stations are selected, the user can see the data reflected on the generated contour map without having to upload their own data manually.

VII.2. NYUS Flow

As listed above, we are not expected to deploy the NYUS model in a user-facing capacity since we only began working on it in Sprint 4, and completing the full frontend implementation is not feasible within the given timeframe. However, we intend to hand it off in a usable state to WSUDAS so that they may deploy it whenever they see fit. We will provide the API implementation and ability to receive the predicted results from the NYUS2.2 model. WSUDAS may then choose how they wish to relay this information to their users. As such, the process for using the model will simply consist of providing sample data through the API and receiving the predictions back from the model.

VIII. Social Responsibility and Broader Impacts

This project contributes to improving agricultural decision-making by providing more accurate and timely data to farmers. By integrating Scoutlabs trap data and NYUS2.2 model outputs, the system enables users to make informed decisions regarding pesticide use, potentially reducing environmental impact, unnecessary chemical usage and having enough information extracted from the weather to prepare for the worst.

From a social responsibility perspective, this aligns with ethical software engineering practices by supporting sustainability and food security. The system improves accessibility to agricultural data and promotes more efficient farming practices.

The project also contributes to broader impacts by extending WSUDAS beyond potatoes to other crops such as grapes, increasing inclusivity across agricultural sectors. Additionally, the system supports future research and student-led development by maintaining a scalable and modular architecture.

IX. Product Delivery Status

The final prototype would be delivered to our project manager Jesse in between April 11th before we all go into further testing. Since the work is done in a private repository, another public repository will be available for documentation and what's done within the project. For building up the project, accessing the private repository is required, but there will be a standalone version of the NYUS 2.2 model as an API endpoint available.

X. Conclusions and Future Work

X.1. Limitations and Recommendations

One obvious limitation during this project is the lack of knowledge on the current framework. This combined with an existing codebase made the kickoff take much longer than expected. This is also a recurring problem as we often than not get stuck when it comes to integrating new developed features into the current codebase. The recommendation for this problem is to fully digest the tools used before doing anything. Another limitation we faced is a bit outside of the scope as we lack the skill to gather required data for the model, but Jesse did his best to help us with useful data for the validating step. Communication was also a problem early on, but learning how to communicate better helps, including when to say we haven't finished our work yet.

X.2. Future Work

There are still a few points that could use polishing in terms of our design, like using more chunked inserts and changing the logic for the model_card check. However, the main functionality is there and the system does what it is expected to do, and the client has indicated that this is not a major priority for the time being.

Our goals for the next semester are to begin working on frontend implementation, specifically the "click on map" feature which would allow users to view detailed data at specified locations. This will be a significant departure from what we have been working on so far as it will involve more JavaScript writing as opposed to solely php, and we still need to clarify details with the client as to what exactly this will look like in its final form. For now, the draft idea is to let the user click everywhere within the contour and it would return the information from the nearest station, but this is subject to change after the kickoff meeting depending on what the client deems appropriate.

XI. Acknowledgements

We would like to thank Parteek Kumar for helping us get in contact with our client, Washington State University Decision Aid System, and for keeping our group organized and on-track for the past two semesters. We would also like to thank David Crowder of WSUDAS for providing us the opportunity to work with the organization and for facilitating contact with their developers. Finally, we would also like to extend enormous thanks to the developers Jesse Trammell and Izzy McCabe of WSUDAS for working with us very closely throughout the entire course of the project, taking time out of their workdays (and sometimes weekends) to provide advice and assistance to us whenever it was required.

XII. Glossary

Entomology - A branch of biology focused on the study of insects.

API - (Application Programming Interface) Protocols that determine how applications interact with one another.

GDP - (Gross Domestic Product) The total value of all goods produced and services within a country.

WSUDAS - Stands for “Washington State University Decision Aid System.” The organization providing DAS’ service, which displays agricultural information based on the organization’s models. “DAS” is commonly used as a shorthand name both internally by its faculty and within this document.

Perlin Noise Generation - A type of pseudo-random visual generation that produces areas of various visual intensity.

Contour Region - In the context of this project, a contour region is WSUDAS’ visual representation of insect populations in a given area. These user-facing contour regions are the main service provided by DAS.

AWS S3 Bucket - A storage object provided by Amazon Web Services which allows users to upload and access data stored in the cloud

Scoutlabs - An agricultural insect trap company. Their traps catch and count insects, identifying them using imaging data and reporting their counts to users. They are the main focus of the data transfer portion of our project.

LT50 Model - An abbreviation of “Lethal Temperature 50.” This is the NYUS2.2 model’s output, which is predicted temperatures that 50% of grape crops will die at. This allows growers to know what temperatures will be detrimental to their grape crops.

XIII. References

- [1] S. Zahniser, "What is Agriculture's share of the overall U.S. economy?," *Economic Research Service*, Dec. 19, 2024. [Online]. Available: <https://www.ers.usda.gov/data-products/chart-gallery/chart-detail?chartId=58270>
- [2] *Decision Aid System*, "About | Decision Aid Systems," *Decisionaid.systems*, 2025. [Online]. Available: <https://decisionaid.systems/about>. [Accessed: Sep. 13, 2025].
- [3] The Codeholic, "Laravel 12 in 11 hours – Laravel for Beginners Full Course," *YouTube*, Oct. 14, 2024. [Video]. Available: <https://www.youtube.com/watch?v=0M84Nk7iWkA>
- [4] C. M. Bishop, P. Carlsson, S. Andersson, D. Mohamad, P. Nyman, and K. Naylor, *Pattern Recognition and Machine Learning*. MTM, 2018.
- [5] R. Touti, "Perlin Noise: A Procedural Generation Algorithm," *rtouti.github.io*, 2025. [Online]. Available: <https://rtouti.github.io/graphics/perlin-noise-algorithm>. [Accessed: Oct. 20, 2025].
- [6] "General Purpose Buckets Overview - Amazon Simple Storage Service." General Purpose Buckets Overview, Amazon, docs.aws.amazon.com/AmazonS3/latest/userguide/UsingBucket.html.
- [7] WSUDAS. 2025. WSU Decision Aid System / das.website. <https://github.com/wsu-das/das.website.git>
- [8] Baiju Muthukadan, "Selenium with PythonSelenium with Python" [Selenium with Python — Selenium Python Bindings 2 documentation](#)
- [9] *Google Earth*, Google, earth.google.com/web/@0,-0.259,0a,22251752.77375655d,35y,0h,0t,0r/data=CgRCAggBQgIIAEoNCP_____wEQAA?authuser=0.
- [10] "Easy & Affordable AI Powered Remote Insect Monitoring." *Scoutlabs*, Scoutlabs, 9 Aug. 2025, <https://scoutlabs.ag/>.
- [11] Hongrui Wang, et al. NYUS.2: an automated machine learning prediction model for the large-scale real-time simulation of grapevine freezing tolerance in North America, *Horticulture Research*, Volume 11, Issue 2, February 2024, uhad286, <https://doi.org/10.1093/hr/uhad28>.

XIII. Appendix A - Team Information

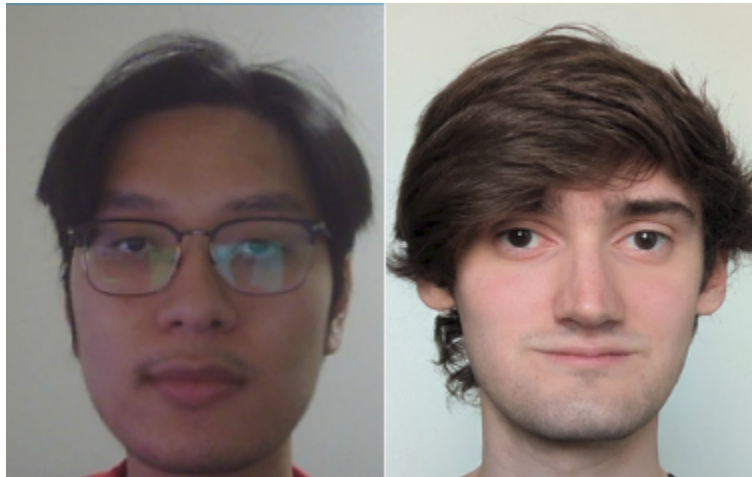


Figure 9: WSUDAS-KS: Son Nguyen (left) and Kieran McHugh (right)

XIV. Appendix B - Example Testing Strategy Reporting

Tested Requirements: Contour Location/Traps transfer:

In our testing we have discovered some hurdles with the system, specifically that multiple traps use the same location tag but are not in fact in the same location. This issue is caused by a trap user owning multiple traps, and thus we had to discover a method to disambiguate between them. We found that there was a unique attribute associated with each trap and decided to use this to differentiate the traps. While this issue has been resolved, we also must change our current implementation to use an update & insert method since this command will be regularly run and must not overwrite existing locations.

Tested Requirements: Scoutlabs Data Transfer

While simply moving data to where it should be was not too much of a challenge, there are additional considerations we need to take into account when writing this function. Specifically, we need to refactor our current transfer to append a “survey_date” value to the entries so that they can be properly and easily used by the user-facing frontend service to display the data in chronological order.

Tested Requirements:NYUS 2.2 Model

For the testing of the new model, it is straightforward compared to the first two as it is an already existing trained model. So the only step to test it out is to validate the output. Still there is a conversion from R to Python for feature extraction, which produces slightly off results (as in the hundreds of the decimal), so extra validation and checking whether it is a rounding problem or miscalculation is a high priority. Integrating it back to the PHP codebase is also a challenge, as now there needs to be some setup for the FastAPI to run as a service.

XV. Appendix C - Project Management

Our team had two weekly meetings with the developer Jesse Tremmell on Mondays and Fridays to report what was worked on during the week and what our focuses would be going forward. Additionally, our team also met with professor Parteek Kumar several times throughout the course of the project to update him on the progress of the project. These meetings were both helpful in their own ways: meetings with Jesse helped to guide us through the daily tasks and challenges presented by the project while meetings with the professor kept us on track on a broader level and ensured that the entire project was on track to completion on a long-term scale. We also used a kanban board to keep track of tasks and took notes from each meeting with Jesse.

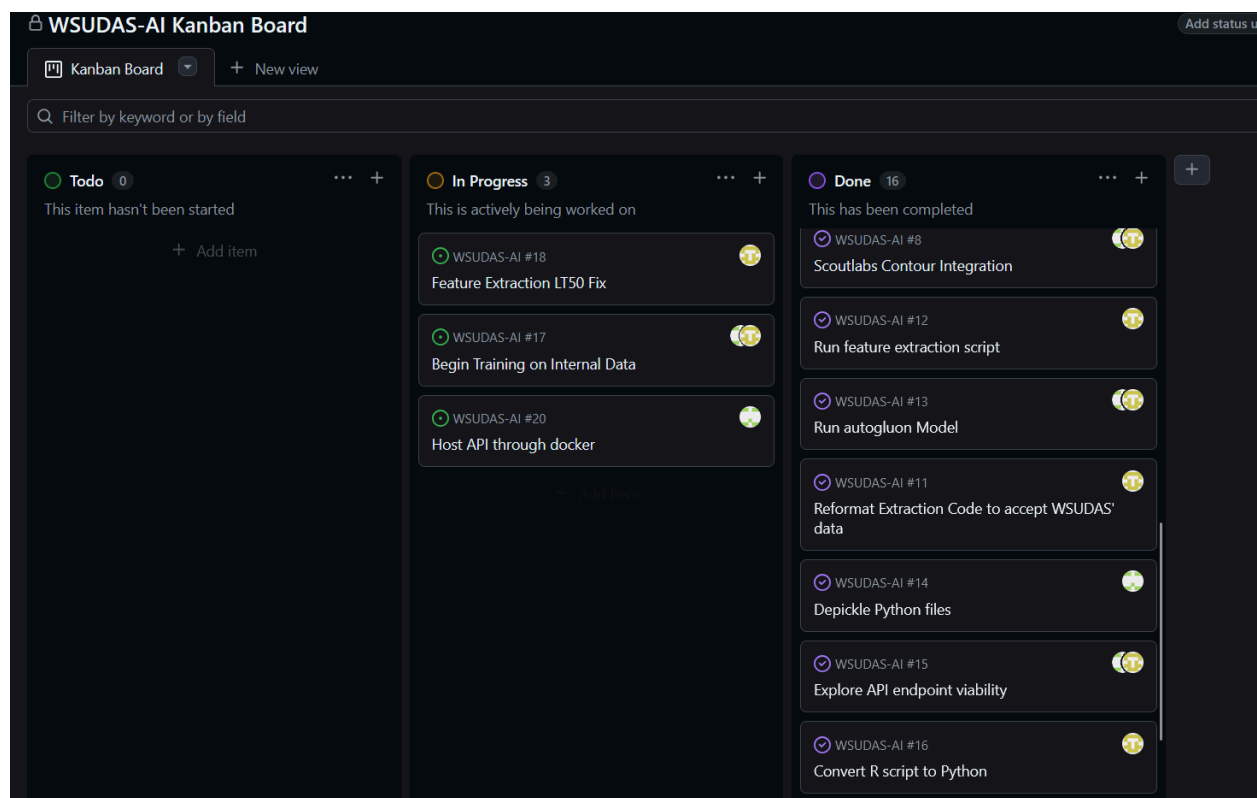


Figure 10: Kanban Board